

SoK: Beyond the Threat Catalog—An Empirical Taxonomy of Authorization Governance and Containment Failures in Model Context Protocol Servers

Anonymous Author(s)
Submission under double-blind review

Abstract—Model Context Protocol (MCP) security literature to date is dominated by two genres: threat-taxonomy surveys that catalog attack classes without grounding them in production code, and large-scale automated scans that measure prevalence of known bug patterns without examining *how* authorization and containment are actually designed. Neither answers what a practitioner needs to know: when a server’s design does attempt to govern what an agent can do, what does that governance look like when it works, and how, specifically and reproducibly, does it fail?

We present an empirical, manually verified audit of 89 distinct public MCP server repositories (91 audit passes across 15 domain-stratified rounds; two repositories were audited twice, see Section III), with every finding graded by evidence tier (code-reading hypothesis, executed-with-mocked-sink, or fully executed proof-of-concept). We find that fewer than 10% of repositories contain an in-scope, exploitable code-level bug (a lower bound, given the scanner’s unmeasured recall); the dominant failure is an un-made authorization decision, not a programming error. We contribute (1) a three-layer authorization taxonomy (control-plane exposure, execution/trust delegation, protocol-level risk signaling) plus an orthogonal implementation-integrity axis; (2) six governance dimensions absent from that taxonomy and from prior MCP literature, including *containment intent without a valid technical primitive*, illustrated by a reproducible sandbox-escape proof-of-concept against a healthcare-record server’s Node.js vm-based “sandbox” — a known-unsafe primitive deployed, without risk annotation, in front of patient data; (3) a catalog of ten mitigation patterns actually observed in deployed code, four of them recurring independently across unrelated codebases; and (4) evidence that a vulnerability’s real-world severity cannot be assessed from server code alone, but depends on platform-level guarantees, hosting opacity, and which party executes a given network operation. We position this work against four concurrent 2025–2026 papers — an existing MCP security SoK, a DSN 2026 registry-scale study (67,057 servers), a 1,899-server empirical study, and a layer-aligned defense taxonomy — none of which contains a comparable governance taxonomy or mitigation-pattern catalog, and we report a partial corpus artifact, a threat model, and an explicit accounting of this study’s methodological limitations.

Index Terms—Model Context Protocol, LLM agents, authorization, access control, agentic AI security, tool-use governance, systematization of knowledge

I. INTRODUCTION

The Model Context Protocol (MCP) standardizes how LLM-driven agents discover and invoke external tools: file systems,

databases, SaaS APIs, payment rails, cryptocurrency wallets, electronic health records. Adoption has been fast and largely uncoordinated — thousands of public servers, written by individual maintainers, official vendor teams, and everything in between, with no central authority reviewing what a given server’s tools are actually authorized to do once an agent decides to call them.

This creates a specific and under-studied risk surface. Unlike a conventional API, where the caller is typically a known, audited client, an MCP tool’s caller is a language model whose decision to invoke a tool can be shaped by content the model was never meant to trust — a webpage it fetched, a message it read, a document it summarized (indirect prompt injection [5]). The security question is therefore not only “does this code have a bug” but “does this server’s design constrain what an LLM-driven caller can do, and does the protocol give the surrounding system (client, policy engine, human) enough information to constrain it further.” We call this second question *authorization governance*, to distinguish it from the vulnerability-class taxonomies (tool poisoning, rug pulls, session hijacking) that dominate existing MCP security literature.

Over 15 domain-stratified audit rounds we manually examined 89 distinct public MCP server repositories, reading source code, running static pattern scans to triage candidate findings, and — for the highest-confidence claims — executing proof-of-concept code against isolated replicas of the exact vulnerable construction. This paper reports the resulting taxonomy, the mitigation patterns we found, and the failure modes we did not expect going in.

Our central empirical claim, sharpened across the full corpus, is that the dominant failure mode in MCP server governance is not a programming bug. Fewer than 10% of the 89 repositories contain an in-scope, exploitable code-level vulnerability (command or sub-language injection, SSRF via an un-overridden library default, or an unsound sandbox). The overwhelming majority of servers with serious governance problems have *zero* programming errors: they simply expose an API’s full control plane as a set of tools, under a single shared credential, with no protocol-level signal distinguishing a read from an irreversible write. This is a design question,

not a code-review question, and it is the reason we frame this work around a taxonomy of governance rather than a catalog of bugs.

A second claim, which required the full 89-repository corpus to see with any confidence, is that positive governance design is not a single “gold standard” pattern that projects either reach or fail to reach. We catalog ten non-interchangeable mitigation patterns — dynamic per-request identity delegation, credentials that never cross the protocol boundary, externally delegated fail-closed policy decisions, auto-derived confirmation gates, persisted blast-radius limiting, and others — each independently recurring in unrelated codebases and domains, none of which by itself covers the full space of what we call Layer 1–3 governance. No repository in our corpus combines more than three or four of the ten.

Third, and this is where we depart most explicitly from the four most closely related 2025–2026 papers we discuss in Section II, severity cannot be read off a server’s source code in isolation. The real-world blast radius of an MCP tool call depends on properties of the platform the server wraps (a QuickBooks “delete” is a soft-delete; an S3 `DeleteObject` is not), on whether the server is self-hosted and auditable or vendor-hosted and opaque, and on whether a given network operation is executed by the MCP server’s own process or proxied to the vendor’s infrastructure. None of the large-scale automated studies we are aware of currently encode this distinction, because it is not visible from static analysis of a single repository.

Contributions.

- A three-layer authorization taxonomy plus an orthogonal implementation-integrity axis, empirically refined against 89 repositories across 15 domains (Section IV).
- Six governance dimensions absent from the original taxonomy and, to our knowledge, from prior MCP security literature: protection against untrusted incoming content, evasion of underlying platform governance, “parametric collapse” of protocol-level risk signaling (a concrete counterexample to blanket “reduce the number of tools” advice, Section V), sub-DSL injection, inconsistent application of an otherwise-correct control, and containment intent without a valid technical primitive.
- Four case studies with graded evidence, including a reproducible proof-of-concept sandbox escape against a healthcare-record MCP server (Section VI).
- A catalog of ten mitigation patterns observed in deployed code, four of them independently recurring across unrelated codebases (Section VII), and a three-dimension severity model for reasoning about real-world blast radius beyond server code (Section IX).
- An explicit positioning against four concurrent works, a partial corpus artifact, and a stated threat model (Sections II–III).

II. RELATED WORK

Foundational security concepts. The core failure this paper documents — a program executing with more authority than

the request that triggered it warrants — is the confused deputy problem [2], here instantiated as an LLM agent acting as an under-scoped deputy between a user’s intent and a tool’s full authority. Indirect prompt injection [5] names the companion risk on the input side: content an agent ingests as data can redirect what it does with its authority. Attribute-based access control and externalized policy decision points (PDP/PEP separation, e.g., Open Policy Agent/Rego) provide the standard architectural vocabulary for the “externalized, fail-closed policy” pattern we catalog in Section VII; sandbox isolation technologies such as gVisor and Firecracker provide the contrast case against which we evaluate the unsound `vm-module` “sandbox” in Section VI.

MCP-specific security literature. Four works published in the six months prior to this writing bear directly on our contribution and require explicit positioning.

An existing MCP security SoK [12] systematizes MCP threats (indirect prompt injection, tool poisoning, alignment failures) and surveys proposed academic defenses (cryptographic tool-definition signing, e.g. ETDI; runtime intent verification; gVisor-based session isolation). It does not report original empirical analysis of MCP server repositories, does not discuss MCP’s native tool annotations (`readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`) as a governance mechanism, and does not contain a catalog of concrete, deployed mitigation patterns — its treatment of human-in-the-loop oversight, for instance, is explicitly theoretical. Our work and this SoK are complementary rather than redundant: theirs systematizes the threat/defense space at the level of academic proposals; ours characterizes what a sample of real, deployed servers actually built. We nonetheless take the existence of a same-genre “SoK” on this exact topic seriously — Section VIII discusses how this constrains our own framing.

A large-scale registry study accepted at DSN 2026 [13] scans 67,057 servers across six public registries with an automated tool (MCPInspect), reporting a two-stage attack-surface model (registry, server, host) and flagging 833 vulnerable servers plus 18 with suspicious tool-description metadata. Its scale is two orders of magnitude beyond ours; its method is necessarily shallower per server, and — as with the SoK above — it does not report on authorization-scoping design, HITL confirmation patterns, or a positive-pattern catalog.

An empirical study of 1,899 open-source servers [14] applies a hybrid static-analysis pipeline (a general-purpose analyzer plus an MCP-specific scanner) and reports eight vulnerability types, 7.2% of servers with general vulnerabilities, 5.5% with MCP-specific tool poisoning, and maintainability figures (66% code smells, 14.4% known bug patterns). Governance mechanisms and positive design patterns are outside its scope.

MCP-DPT [15] organizes MCP-relevant attacks and proposed defenses by the architectural layer responsible for enforcement, reporting that defensive coverage in the literature is uneven and concentrated at the tool layer, with gaps at host orchestration, transport, and supply-chain layers. It is, like the

Dimension	[12]	[13]	[14]	[15]	Ours
Original empirical repo audit	–	✓	✓	–	✓
Tool-annotation governance	–	–	–	–	✓
HITL confirmation patterns (deployed)	–	–	–	–	✓
Positive mitigation-pattern catalog	–	–	–	–	✓
Platform-dependent severity analysis	–	–	–	–	✓
Executed proof-of-concept exploit	–	partial	–	–	✓
Threat/defense literature survey	✓	–	–	✓	partial
Registry-scale prevalence statistics	–	✓	✓	–	–

TABLE I: Coverage of governance-relevant dimensions across the four most closely related works and this paper, based on full-text or full-abstract review of each (Section II).

SoK, a literature-mapping contribution rather than an empirical study of deployed server code.

Positioning. All four works either (a) survey the threat/defense literature without auditing deployed servers, or (b) audit deployed servers at large scale for known vulnerability classes without examining authorization-governance design or cataloging positive patterns. Table I makes this concrete rather than asserted: for each of our taxonomy’s four layers/axes and the six governance dimensions of Section V, we mark whether each of the four works covers it at all. None reports a manually-verified, evidence-graded account of *how* authorization and containment succeed or fail in deployed code, and none contains anything resembling our mitigation-pattern catalog. This is the gap this paper fills — at a scale two orders of magnitude smaller than the largest automated study, but with a correspondingly deeper per-repository analysis, including one proof-of-concept exploit that automated scanning of this style would not have surfaced as an executable finding.

III. METHODOLOGY

Corpus. 91 audit slots across 15 rounds, stratified by application domain rather than by registry popularity: infrastructure/CLI/containers (round 1), local-execution risk surfaces — filesystem, Docker, Android, CLI (round 2), databases: SQL/NoSQL/analytics (round 3), smart home/identity/cloud infrastructure (round 4), secrets/payments/crypto/email (round 5), identity-IAM/messaging (round 6), crypto wallets/CI-CD/IaC (round 7), productivity/vector databases (round 8), media and document processing/observability (round 9), social networking (round 10), cloud storage/browser automation (round 11), search/scraping/third-party actor execution/maps (round 12), CRM/support/e-commerce (round 13), personal and business finance (round 14), and health-care/legal/HR/scheduling (round 15). Rounds 1–5 varied in size (31 repositories total); rounds 6–15 fixed at six repositories each (60 total). Assembling the corpus appendix for this draft (Table V, Appendix A) surfaced two bookkeeping errors from the original 15-round campaign, which we report explicitly rather than silently correct away, since they are the kind of error a hand-run, multi-session audit is prone to and that we would rather flag ourselves than have a reader find. First, one repository (an identity-management server) was queued in round 4 but could not be audited there because our scanner did not yet support its language; it was re-

queued and fully audited in round 6. Second, one repository (a spreadsheet-integration server) was examined in round 3 and, independently, selected again as a candidate in round 8, with the same finding (a deliberately-committed, scope-limited test credential) recorded twice under slightly different framing. Counting both repositories once each, the corpus comprises **89 distinct repositories** audited across 91 round-slots. Table V lists the full corpus by round, star count, and language, with five repositories’ names withheld pending completion of the responsible-disclosure process described in Section X.

Within each domain, candidate repositories were selected opportunistically — search results, official vendor repositories, and repositories referenced from MCP ecosystem directories available at audit time — not via a systematic registry crawl. **We flag this explicitly as a limitation, not a strength:** it is a convenience sample, and we make no claim that it is statistically representative of the broader MCP ecosystem. Section XI discusses what a registry-scale sampling frame, as used in [13], [14], would add. The taxonomy in Section IV was stable across all 15 rounds; the six additional dimensions in Section V and the ten-pattern catalog in Section VII were named retroactively as they recurred, meaning later rounds were explicitly checked against dimensions that earlier rounds’ write-ups could only have noted informally. We did not perform a formal re-coding pass of rounds 1–5 against the finalized six-dimension/ten-pattern rubric; where an early-round repository is cited against one of those dimensions in this paper, it is because the original round write-up already described the relevant behavior in enough detail to support the classification, not because of a later re-audit. This is an instrument-stability limitation we surface rather than paper over.

Threat model. We assume an adversary who cannot authenticate directly to the backend service a given MCP server wraps, but who can influence the decisions of the LLM agent invoking that server’s tools — through indirect prompt injection via content the agent ingests (a fetched page, a summarized document, a message it reads) [5]–[7], through a compromised or malicious upstream MCP server in a multi-server session, or through an under-specified user request that a permissive tool surface lets the agent over-interpret. Under this model, a tool’s danger is a function of what it lets an *agent acting on attacker-influenced input* do, not of what the authenticated human principal could already do through another interface. This matters concretely for the Salesforce case in Section VI-B: that a logged-in Salesforce admin could open the Developer Console and run the same Apex code is not in dispute and is not the finding. The finding is that the MCP tool surface offers no signal distinguishing this capability from the server’s seven read-only, permission-respecting tools, so an agent whose behavior has been steered by untrusted content has no protocol-level obstacle, and the human operator supervising it has no protocol-level warning, before that capability is invoked. We apply this same lens — capability reachable by an adversarially-influenced agent, not capability reachable by the authenticated user through any

means — to every case study in Section VI.

Triage. Each repository was cloned and run through a static pattern scanner covering command/shell injection, eval/dynamic code construction, hardcoded secrets, unsanitized path construction, and language-specific equivalents (Go, Rust, Java, C#) for the same classes, plus SQL-injection patterns for database-facing servers. Every match was manually triaged; roughly 90% of scanner hits across the campaign were false positives (test fixtures, safe API usage, substring collisions such as the word “Updated” matching a SQL-verb pattern) and are reported as such rather than discarded silently. The scanner’s false-positive rate is measured; its false-negative rate is not, since we have no independent ground truth for this corpus. We therefore treat every scanner-derived prevalence figure in this paper (Section IV’s “fewer than 10%”) as a **lower bound on detected code-level bugs**, not a prevalence estimate — if anything this strengthens our central claim, since a floor this low is already sufficient to establish that governance failure, not code defects, dominates the corpus.

Evidence tiers. We distinguish, and report per finding, three tiers of confidence, defined by what was actually executed rather than by subject matter: (1) *code-reading only* — the vulnerable path was traced and cross-referenced against documentation (e.g., a platform’s documented execution-context semantics) but no code was run; (2) *executed with the final sink mocked* — we ran the real, unmodified vulnerable construction (command/argument assembly, sandbox setup) end to end, intercepting only the final external effect (a process spawn, a network call) to avoid touching a live target, and captured the concrete output that effect would have received; (3) *fully executed* — we ran the complete vulnerable construction, including the final effect, against an isolated reconstruction (never the live target). Section VI reports one tier-3 finding (Section VI-A), one tier-2 finding (Section VI-C), and two tier-1 findings (Sections VI-B and VI-D). We consider this tiering itself a methodological contribution: a taxonomy that does not distinguish these tiers risks presenting a code-reading hypothesis with the same rhetorical weight as an executed exploit — and, as tier 3 in Section VI-A illustrates, even “fully executed” needs a further caveat: executed against a faithful reconstruction is not the same claim as executed against the artifact, and we flag that distinction explicitly where it applies.

Single-reviewer verification. All triage and verification in this corpus was performed by a single reviewer (the author). We did not run an independent second-reviewer pass or an adversarial re-verification step, and we do not report inter-rater reliability, because there is only one rater. This is a material limitation for venues that expect it on vulnerability classification; Section XI discusses what a second-coder pass on a stratified subsample would add, and we consider it the single highest-value addition available before a top-tier submission. Much of the mechanical execution of this process — reading candidate repositories, running and triaging the scanner, constructing and executing proof-of-concept code, and drafting round-by-round write-ups — was carried out by an AI research assistant working under the author’s direction;

every finding and every claim in this paper was reviewed and approved by the author against the underlying code (and, where applicable, executed PoC output) before being recorded. “Single reviewer” denotes human review authority over the corpus, not unassisted manual execution of every step; the LLM Usage Considerations section gives a full account of this division of labor.

Ethical handling. No proof-of-concept was executed against a live third-party service, a populated database, or real user data. The sandbox-escape PoC in Section VI-A was reconstructed against a synthetic placeholder object in an isolated local process, replicating only the vulnerable code construction, never the target repository’s live deployment or any patient data. Section X states our disclosure policy and the status of each reported finding at the time of writing.

IV. A LAYERED TAXONOMY OF MCP AUTHORIZATION GOVERNANCE

Refined across the full 89-repository corpus, we retain a three-layer structure plus one orthogonal axis, summarized in Table II.

Layer 1 failure is both the most common and most severe class in the corpus. A home-automation server exposing a single `call_service_tool` that is the entire control surface of a smart home, or a secrets-management server mapping all 24 administrative operations of its backing vault 1:1 onto MCP tools, are examples with no code bug anywhere — the servers do exactly what their source says, and no one decided they needed a boundary. Approximately 8 of the 89 repositories (under 10%) contain an in-scope, exploitable programming error; the rest of the corpus’s serious findings are governance decisions, not defects.

Layer 2 failure is best measured along two independent sub-axes: how authority is delegated (a single shared credential versus per-request, per-user delegation) and whether high-impact mutations require confirmation. The strongest pattern we observed for the former — a credential that never crosses the MCP protocol boundary at all, discussed in Section VII — was found in exactly one repository across the entire corpus.

Layer 3 remains the layer where the original taxonomy holds with the least revision: of 89 repositories, only approximately 10 (roughly 11%) use MCP’s native risk annotations systematically rather than as an isolated, one-off case. It remains the exception rather than the norm, though the fuller corpus now supplies enough examples, across Go, TypeScript, Python, C#, and Java codebases, to characterize what correct usage looks like in practice rather than asserting only its absence.

V. GOVERNANCE DIMENSIONS BEYOND THE ORIGINAL TAXONOMY

Six patterns recurred reproducibly — five of them in two or more unrelated repositories — without fitting cleanly into Layers 1–3 or the orthogonal axis as originally defined.

(1) Protection against untrusted incoming content. The three layers govern what a tool can *do*; none governs what

Layer	Question it answers	Characteristic failure
1. Control-plane exposure	Which part of the underlying API surface is exposed as a callable tool?	1:1 mapping of the full underlying API (data plane <i>and</i> control plane) onto MCP tools, with no reduction for least privilege.
2. Execution and trust	Under what identity, and with what confirmation, does a tool actually execute?	Blind delegation of authority (a single shared, static credential with no per-invocation scoping) combined with no human-in-the-loop gate on high-impact mutations.
3. Protocol-level risk signaling	Does the server tell the MCP client how risky a given tool is?	Omission of native risk annotations (<code>destructiveHint</code> or equivalent), leaving the LLM as the only point of self-restraint.
Orthogonal: implementation integrity	When a boundary <i>is</i> attempted, is it correctly built?	A bug in the containment mechanism itself — not absent design, but design that does not deliver the guarantee it claims.

TABLE II: The core taxonomy, stable in structure from the first five (of 15) audit rounds (31 repositories) but substantially deepened by the full 89-repository corpus.

enters the LLM’s context before it decides. Exactly one repository in the corpus, a Telegram client server, explicitly sanitizes incoming message and chat-name content and annotates it as “user-audience data, not instructions,” directly operationalizing the indirect-prompt-injection threat model of Greshake et al. [5]. No other repository in the corpus does this.

(2) Evasion of underlying platform governance. A Slack-integration server can optionally authenticate via browser session tokens rather than a registered OAuth application. This is not a bug; it is a documented, deliberate feature. Its governance implication is that the workspace owner’s installed-apps audit — the platform’s own mechanism for knowing what has access — never records this access, because it never registers as an app. No framework we are aware of that reasons only about “what does the MCP server do” names this class of vector, because the failure is one level up, in how the server represents itself to the platform it wraps.

(3) Parametric collapse. An identity-management server collapses 178 distinct administrative operations behind a single MCP tool, `executeKeycloakOperation(operation, params)`, where `operation` is a string parameter. This is not an oversight in annotating tools; it is a deliberate response to API-surface bloat that, as a side effect, destroys the one place the protocol allows distinguishing “read a user” from “delete an entire realm”: the tool name itself. It is a direct counter-example to any blanket recommendation to “reduce the number of tools,” since reducing tool count and preserving per-operation risk granularity can be in direct tension.

(4) Sub-DSL injection. Established independently in two unrelated FFmpeg-wrapping repositories (one building commands via string interpolation plus `shlex`, the other via `argv` lists) and reinforced by a third case (a YouTube-transcript tool piping URLs to `yt-dlp`). When the invoked binary has its own sub-expression language — FFmpeg’s `filtergraph` syntax is the case we verify in Section VI-C — an unsanitized value can inject new nodes into that language even when process invocation itself is textbook-correct (`argv` as a list, no shell).

This is a distinct sub-type of implementation-integrity failure: the correct boundary sits one level below where the developer placed it.

(5) Inconsistent application of an existing, correctly-built control. Distinct from “opt-in mechanism, permissive default” (the most common Layer 1/2 failure in the corpus). Here the control is mandatory, well-built, and tested — but does not cover the full surface it should. Section VI-D presents this case in detail: a three-layer SSRF defense covering four of five tools in one file, bypassed entirely by the fifth.

(6) Containment intent without a valid technical primitive. The most severe and most rigorously verified finding of the campaign, detailed in Section VI-A: a healthcare-record server’s use of Node.js’s `vm` module as a documented “sandbox” for evaluating arbitrary user-supplied JavaScript against a full patient record, despite Node’s own documentation stating unambiguously that `vm` is not a security mechanism. Distinct both from “no containment was attempted” and from sub-DSL injection: here the code explicitly declares an intent to sandbox, and the primitive chosen does not deliver that guarantee — a gap between believed and actual containment that is, we argue, the cleanest single case in the corpus for the claim that auditing MCP governance cannot stop at “does a mechanism exist” and must ask “does the chosen primitive actually provide the guarantee the code assumes it provides.”

VI. CASE STUDIES

We present four case studies spanning the evidence tiers defined in Section III, chosen because each anchors a distinct point in the taxonomy above.

A. Tier 3: sandbox escape in a healthcare-record server

A public healthcare-record MCP server (name withheld pending disclosure; see Section X) loads a patient’s complete FHIR bundle (conditions, medications, clinical notes, lab results) into memory and exposes a code-evaluation tool for ad-hoc analysis over that record. Its implementation:

```
const sandbox = { fullEhr, console: sandboxConsole,
  _: _, Buffer: Buffer,
  _resultPromise__: undefined };
const scriptCode =
```

```
'async function userFunction(fullEhr, console, __, Buffer)
{
  "use strict"; ${userCode}
}
__resultPromise__ = userFunction(fullEhr, console, __,
  Buffer);`;
const context = vm.createContext(sandbox);
const script = new vm.Script(scriptCode,
  { filename: 'userCode.vm' });
script.runInContext(context,
  { timeout: timeoutMs / 2, displayErrors:
    true });
```

`userCode` is a string supplied directly by the MCP tool caller. Node.js’s own documentation states plainly: “the `vm` module is not a security mechanism. Do not use it to run untrusted code” [10]. `vm.createContext` isolates objects that originate *inside* the sandbox, but not objects the host code itself passes *into* it as parameters — here, three: `console`, a `lodash` reference, and, critically, `Buffer`, all constructed in the host process’s realm. This is the same class of escape documented in multiple disclosed CVEs against the (now-deprecated) `vm2` sandboxing library [9]; we did not discover a novel technique, and we say so plainly rather than implying otherwise — our contribution here is not the exploit but the observation that a primitive Node’s own maintainers, and a well-documented CVE history, describe as unsafe was deployed with no risk annotation directly in front of full patient records, by a project positioned as reference-quality within the ecosystem it targets.

We reconstructed this exact sandbox construction in an isolated script, using only a synthetic placeholder data object (no real patient data, and never against the target repository’s live deployment), and supplied `userCode` that never references `fullEhr` at all:

```
const hostFunction = Buffer.constructor;
const hostProcess = hostFunction('return process')();
return {
  escaped: true,
  pid: hostProcess.pid,
  cwd: hostProcess.cwd(),
  someEnvKeys: Object.keys(hostProcess.env).slice(0, 5),
  platform: hostProcess.platform,
};
```

Running this against the reconstructed sandbox produced:

```
{
  "escaped": true,
  "pid": 41522,
  "cwd": "[reconstruction working directory,
    not the target server's]",
  "someEnvKeys": ["<names withheld -- reconstruction
    ran in an unrelated sandboxed
    development environment, not the
    target repository's runtime>"],
  "platform": "linux"
}
```

The “sandboxed” code obtained a live reference to the host process object — not a copy. In our reconstruction this exposed only the reconstruction environment’s own, unrelated process state; we did not run this against the target repository’s live deployment, and we do not know what its production environment variables hold. We report this as what it is: proof that the escape technique succeeds against the exact sandbox-construction code the target repository ships, not as

evidence of what a real deployment’s environment contains. The escape does not depend on any timeout-evading technique: it is synchronous and completes in microseconds, well under the 2.5-second execution budget the server itself enforces. The evaluation tool carries no risk annotation of any kind, alongside the server’s other four tools — nothing at the MCP protocol level signals that this is, in practical effect, an arbitrary-code-execution primitive against the server’s own process, rather than a scoped data-analysis tool confined to one patient’s record.

This is, to our knowledge, the only tier-3 finding in the corpus, and the most severe by target domain: healthcare data, a vulnerable primitive well-documented as unsound by its own maintainers and by prior CVE history [9], and an exploit requiring no unusual sophistication.

B. Tier 1: absent control-plane governance in a CRM platform integration

A Salesforce MCP server (name withheld pending disclosure) exposes fourteen tools; seven read-oriented tools are correctly annotated `readOnlyHint: true` and route through Salesforce’s standard REST API, which enforces field-level security and sharing rules for the authenticated user. An eighth tool, `salesforce_execute_anonymous`, takes a string of Apex code (Salesforce’s Java-like proprietary language) directly from the caller and executes it via `conn.tooling.executeAnonymous(args.apexCode)`. Anonymous Apex execution runs, by default, in *system mode*: unlike the REST API path the server’s own read tools use, field-level security and sharing rules do not apply unless the Apex code itself explicitly invokes them (`WITH SECURITY_ENFORCED`, `Security.stripInaccessible`) — a requirement that cannot meaningfully be imposed on an arbitrary string supplied by an external caller. The tool’s own description actively invites this use: “This tool can be used for data operations or updates when there are no other specific tools available. [...] this tool can be used if the operation is achievable using Apex code.” The same server also exposes `salesforce_write_apex/salesforce_write_apex_trigger`, which persist Apex classes and triggers that continue executing after the agent’s session ends. None of the three tools carries any risk annotation or gating mechanism.

We classify this tier 1 (confirmed by code reading and cross-referenced against Salesforce’s documented Apex execution-context semantics, not executed against a live org). Under the threat model in Section III, the finding is not that an authenticated Salesforce admin can run privileged Apex — they already can, through the Developer Console, and that capability existing is not in dispute. The finding is that this MCP server’s tool surface gives an agent whose behavior may be steered by untrusted content, and a human operator supervising that agent, no protocol-level signal distinguishing this tool from the seven permission-respecting read tools beside it: no annotation, no gate, and a tool description that actively

invites its use as a general-purpose fallback. We treat it as the corpus’s clearest example of a tool whose explicit purpose is to bypass the authorization model that governs every other tool in the same server — a qualitatively different severity class from an unannotated ordinary write tool, and a case against treating “missing annotation” findings as interchangeable in severity.

C. Tier 2: sub-DSL injection in FFmpeg-wrapping servers

Two independently-authored servers (names withheld pending disclosure) wrapping FFmpeg exhibit the same vulnerability class through different implementation strategies. The first builds a command via string interpolation:

```
x = f"{dx}"; y = f"{dy}"
cmd = (f' -i {shlex.quote(background_video)} '
      f'-i {shlex.quote(overlay_video)} '
      f'-filter_complex "[0:v][1:v]overlay=x={x}:y={y}[ov]';...")
```

with `dx/dy` untyped at the MCP tool boundary. We executed the real command-construction function (mocking only the final process call) with `dx = '0[ov2];movie=/etc/passwd[leak]'`; `[leak]format=yuv420p'`, confirming the resulting `-filter_complex` value closes the original overlay node and injects an entirely new filtergraph node, `movie=/etc/passwd[leak]` — FFmpeg’s movie filter opens arbitrary local files (and, in network-enabled builds, arbitrary URLs) as a video source. The second server builds `argv` as a list — the textbook-correct mitigation for shell injection — yet exhibits the identical class of bug: `cmd.extend(["-vf", f"scale={resolution.replace('x',' : ')}"])` with `resolution` unchecked against a numeric pattern, allowing an injected comma-separated filter node regardless of the `argv`-list construction. Both are tier-2 findings: we executed the real, unmodified command-construction code end to end with only the final `Popen/subprocess.run` call mocked, capturing the exact `argv/command` string that would have been passed to FFmpeg.

The finding matters beyond the two instances: it demonstrates that “do not use a shell” — the standard, and largely sufficient, mitigation for classic command injection — is necessary but not sufficient whenever the invoked binary has its own sub-expression language. We are not aware of this sub-type being named as distinct from command injection in prior MCP security literature.

D. Tier 1: inconsistent application of a correctly-built control

A general-purpose web-fetching server (name withheld pending disclosure) implements what is, by corpus standards, an unusually complete SSRF defense for four of its five tools: URL-string validation, DNS-resolved-IP validation, and re-validation after HTTP redirects, backed by dedicated tests. Its fifth tool, a YouTube-transcript fetcher, accepts any well-formed URL (`url: z.string().url()`, with no domain restriction) and passes it directly to `yt-dlp` via `execFileSync`, a tool which — for unrecognized

sites — falls back to a generic extractor that itself issues HTTP requests to the given URL, entirely outside the file’s own `validateUrl/validateResolvedIp` functions. We classify this tier 1 (confirmed by code reading — the schema lacks a domain restriction and `execFileSync` receives the raw URL without ever invoking the validation functions used elsewhere in the same file; we did not execute `yt-dlp` against an internal endpoint, judging it inappropriate for this environment). The distinguishing feature versus the “opt-in mechanism, permissive default” pattern common elsewhere in the corpus is that no operator misconfiguration is required here — a new tool merely needs to be added to the same module without routing through the existing validation function.

VII. A CATALOG OF REAL-WORLD MITIGATION PATTERNS

With a single positive example (a cryptocurrency-exchange server with explicit read/write API separation) found in the first five audit rounds, that early stage of the corpus could report only that correct governance *existed* as an exception. The full 89-repository corpus supports a catalog of ten patterns (Table III), four of them independently recurring across unrelated codebases and six observed once; none is by itself sufficient. In our qualitative review no single repository combined more than three or four of the ten — we report this as a qualitative impression from reading all 89 repositories, not as an exact count, since we did not run a systematic per-repository tally against all ten patterns (only the pattern-level citations in Table III were tracked precisely).

A recurring meta-pattern is worth naming on its own: *when a project builds the full mechanism, it tends to ship it disabled by default*. We observed this independently in three unrelated domains: an identity-administration server (`mrz1880/mcp-keycloak-admin`) with real elicitation-based confirmation and allow-listing, both opt-in via environment variables defaulting to permissive; a workspace-productivity server (`awkoy/notion-mcp-server`) with a fully-implemented allow-list/block-list and read-only mode, all opt-in; and a healthcare-record server’s field-level response filter (`wso2/fhir-mcp-server`), which returns the full unfiltered resource unless the caller explicitly requests a field subset. This suggests that maintainers who invest in building correct governance machinery are, separately, making a usability trade-off that quietly undoes part of that investment for the default configuration most deployments will actually run.

VIII. DISCUSSION: WHAT DETERMINES REAL-WORLD SEVERITY

Two findings held up, independently, across multiple domains once the corpus reached sufficient scale and would not have been visible in a single-domain or small-sample study.

Adoption does not predict governance quality — a hypothesis supported by consistent anecdote, not yet by a corpus-wide statistic. The best-governed repositories in the corpus (an identity-administration server implementing all

Pattern	Description and recurrence
1. Dynamic identity delegation	Calling user’s own token, not a shared credential, forwarded to the backend, which decides. <code>sshaaf/keycloak-mcp-server</code> , <code>hashicorp/terraform-mcp-server</code> — both server-enforced.
2. Credential never crosses the protocol boundary	Caller gets only an opaque alias; the real credential (a bank-linking token) is never a tool argument, encrypted at rest with a fallback key hierarchy. <code>arjabbar/PlaidMCP</code> ; strongest Layer-2 pattern observed.
3. Externally delegated, fail-closed policy decision	A pre-execution webhook to an external policy endpoint gates high-value payments; on time-out/unrecognized response, default is deny. <code>lightningfaucet/lightning-wallet-mcp</code> , an independent PDP/PEP separation.
4. Auto-derived confirmation, not opt-in	Any tool registered with a destructive-risk annotation is automatically gated for human confirmation. <code>helbertparanhos/cloudflare-mcp-pro</code> .
5. Persisted, caller-independent blast-radius limiting	Server-side budgets, circuit breakers, rate limits a compromised agent cannot disable via any parameter. <code>devag7/linkedin-mcp</code> (social posting), <code>hifriendbot/agentwallet-mcp</code> (per-agent payment budgets).
6. Delegated verification via the real engine	Asking the backend engine itself (e.g., is this SQL read-only) instead of re-implementing a parser with regex. <code>ergut/mcp-bigquery-server</code> .
7. Secure by default, explicit opt-in to relax	Dangerous configuration exists but needs explicit operator action. <code>microsoft/playwright-mcp</code> (unrestricted filesystem/file:// access), <code>alpacahq/alpaca-mcp-server</code> (real-money vs. paper trading).
8. Honest annotation on the most dangerous tool	A tool that “only orchestrates” third-party code is annotated destructive/open-world, not just obvious CRUD verbs. <code>apify/actors-mcp-server</code> .
9. Sanitization of untrusted incoming content	Incoming messages sanitized and marked as data, not instructions — addresses indirect prompt injection. <code>chigwell/telegram-mcp</code> .
10. Full SSRF pinning	Range-blocking plus pinning the connection to the validated IP, not just resolving it. <code>hifriendbot/agentwallet-mcp</code> (payments), <code>cameronrye/activitypub-mcp</code> (federation) — independently convergent.

TABLE III: Ten independently-recurring, non-interchangeable mitigation patterns found across the 89-repository corpus. No repository combines more than 3–4.

three layers with interactive confirmation; a bank-aggregation server with the strongest credential-isolation pattern found) have 0–1 GitHub stars. Some of the highest-adoption repositories (a messaging integration with 1.8k stars, an infrastructure-as-code server with 1.5k stars) are solid but not exemplary, and several of the corpus’s weakest governance cases include a moderation bot with 468 stars and two officially vendor-maintained accounting-integration servers. We present this honestly as a pattern that recurred, unprompted, in essentially every round of the audit that included at least one high-star repository — not as a population-level correlation. We did not compute a governance score for all 89 repositories against star count (e.g., a rank correlation), and the convenience-sampling limitation in Section XI applies here with particular force: our domain strata were not chosen with star-count balance in mind, so an aggregate statistic computed on this specific corpus would itself be of uncertain external validity. We flag the systematic version of this test as concrete, low-cost future work (Section XI).

Severity is not a property of the server’s code alone. Three independent cases establish this: an accounting-integration server’s unannotated `delete_*` tools operate against a backend where “delete” is a reversible soft-delete, materially reducing real-world blast radius versus an equivalent tool against an irreversible backend operation; a trading server defaults to a simulated paper-trading environment, so the same unrestricted-order tool carries categorically different risk before and after an explicit operator opt-in; and search/scraping servers that proxy fetches to vendor-side infrastructure change which network an eventual SSRF would

actually reach, compared to a functionally similar tool that performs the fetch from the MCP server’s own host. We also encountered one MCP integration distributed with no local code at all — a vendor-hosted server reachable only via OAuth against infrastructure the auditor cannot inspect — which we note as a structural limit on what static analysis of public repositories can characterize about the ecosystem: the self-hosted, auditable subset is not the whole ecosystem, and the vendor-hosted-and-opaque subset is, by construction, outside the reach of the method used here and in the two large-scale automated studies discussed in Section II. Section IX formalizes this observation.

IX. A SEVERITY MODEL BEYOND SERVER CODE

Section VIII argues that a tool’s real-world severity cannot be read from the server’s own code. We formalize this along three dimensions and apply it to the four case studies of Section VI, to make the argument checkable rather than anecdotal.

Platform reversibility. Does the backend the tool ultimately calls treat the operation as recoverable (a soft-delete, a cancelable order, a moderated post) or as destructive (a hard delete, an irreversible on-chain transaction, a broadcast message)? This is a property of the wrapped platform’s API contract, not of the MCP server’s code.

Hosting transparency. Is the server self-hosted and auditable by the operator (the overwhelming majority of this corpus), or vendor-hosted and opaque, where the operator’s only visibility is a README and an OAuth flow? Opacity does not imply insecurity, but it does mean severity cannot

be assessed by the method used in this paper, or in the two large-scale automated studies discussed in Section II.

Execution locus. For a tool that makes an outbound network request, does that request originate from the MCP server’s own process (and therefore its own network position), or is it proxied to vendor-side infrastructure? This does not change whether a request-forgery flaw exists, but it changes which network is exposed if one is exploited.

Applying the model to our own case studies is, admittedly, less illuminating than applying it across the corpus, since all four case studies happen to be self-hosted, server-local findings — which is itself informative: it suggests our case-study selection, driven by “most severe verified finding per dimension,” was not neutral with respect to hosting and locus, both of which are harder to audit in a vendor-hosted or vendor-proxied server. A natural extension of this work is to score the full corpus (not just the four headline cases) along these three dimensions and report the joint distribution, which we leave to future work alongside the registry-scale sampling frame discussed in Section XI.

X. ETHICS AND RESPONSIBLE DISCLOSURE

Review. This audit was conducted by a single researcher examining public source code and running proof-of-concept code only against private, isolated reconstructions (never live third-party targets); no human-subjects data was collected and no live system was accessed without authorization. We did not seek institutional ethics-board review, judging it not applicable under our institution’s policy for static analysis of publicly available source code with no human-subjects component; we note this determination explicitly rather than silently, so a reader can disagree with it.

No live exploitation. No proof-of-concept in this paper was executed against a live third-party deployment, a populated production database, or real user data. The sandbox-escape reconstruction (Section VI-A) used a synthetic placeholder object in an isolated local process. The FFmpeg findings (Section VI-C) executed the real command-construction code with only the final process-spawn call mocked. The CRM (Section VI-B) and web-fetch (Section VI-D) findings were verified by code reading alone, specifically because we judged live execution inappropriate against those targets.

Disclosure policy. For each finding in Section VI we intend to notify the maintainer(s) through the repository’s stated security channel where one exists, or a private issue/email to the listed maintainer otherwise, with a 90-day remediation window before any public artifact (including a future version of this paper) names the affected repository or publishes exploit code for it. Where a maintainer does not respond within that window, we will follow GitHub’s and CERT/CC’s standard coordinated-disclosure guidance for unresponsive maintainers rather than unilaterally shortening or extending the window.

Status at time of writing, per finding. Of the five affected repositories (four case studies in Section VI; the sub-DSL injection case study covers two independently-authored repositories), two were formally reported on 2026-09-04 through

GitHub’s private vulnerability reporting mechanism — the sub-DSL injection finding against the first FFmpeg-wrapping server (GHSA-v8gr-ccqj-pp2g) and the SSRF-bypass finding against the web-fetch server (GHSA-5285-gc3j-36ff) — both currently in maintainer triage with no response yet received. Of the remaining three, one project’s published security policy names the same GitHub mechanism, but the feature was found disabled at the repository level when we attempted to use it; one has no published security contact and the mechanism is not enabled; one repository has since been archived, which disables GitHub’s reporting mechanism entirely. For these three we are pursuing the best available fallback per repository (a minimal public issue requesting a private contact point, or a maintainer’s linked public profile, with no technical detail included) rather than treating the absence of a formal channel as a reason to stop. All five repository names remain withheld from this draft (including its appendix, Appendix A) regardless of channel status. Consistent with the standard described in Section XI — disclosure complete, or substantially underway with a documented, good-faith attempt through the best channel available for each finding — we treat that bar, not full completion or confirmed remediation for all five, as the practical prerequisite for submission; this section will be updated with maintainer responses and remediation status as they arrive.

The healthcare-record finding specifically. Because the affected server processes FHIR patient data and the finding is a full sandbox escape, we treat it as warranting more than routine maintainer notification. Before any wider distribution of this paper we will (a) attempt to determine, from public information only, whether the project is used in any production clinical deployment, and (b) if so, consider coordinating through a body such as CERT/CC or a relevant health-sector ISAC in addition to direct maintainer contact, rather than relying on maintainer notification alone. This assessment had not been completed at the time of writing.

Publishing exploit code. We include the sandbox-escape technique (Section VI-A) in this draft because it is a previously well-documented technique class (not a novel exploit) and because the draft itself is not yet public — but a public version of this paper released before the disclosure window closes should withhold the copy-pasteable payload and describe the technique in prose only, consistent with standard coordinated-disclosure practice. We flag this as an open action item rather than treating the current draft’s content as ready for release as-is.

XI. LIMITATIONS

Sampling. Repositories were selected opportunistically by domain, not via a systematic registry crawl. We cannot claim the corpus is statistically representative of the MCP ecosystem, and our prevalence figures (e.g., “under 10% of repositories contain an in-scope code bug”) are manual counts over a curated sample, not measurements with a computable margin of error. A registry-scale sampling frame, as used by [13], [14], would materially strengthen any claim of prevalence; it

Case study	Reversibility	Hosting	Locus
Sandbox escape (§VI-A)	irreversible (process compromise)	self-hosted	N/A
CRM anonymous execution (§VI-B)	irreversible (system-mode writes)	self-hosted	N/A
FFmpeg sub-DSL injection (§VI-C)	irreversible (local file read)	self-hosted	server-local
Web-fetch inconsistency (§VI-D)	N/A (read-only)	self-hosted	server-local

TABLE IV: The severity model applied to the four case studies. All four are self-hosted and server-local by construction of our sample (we did not select a vendor-hosted or vendor-proxied case study for Section VI); the model’s discriminating power is illustrated instead by the corpus-wide contrasts in Section VIII (soft-delete accounting tools, paper-trading defaults, vendor-proxied search/scraping).

would not, on its own, replace the manual governance-design analysis this paper contributes, but the two approaches are complementary and a natural extension of this work.

Single-reviewer verification. All classification and verification was performed by one reviewer without an independent adversarial re-check. We mitigate this partially through the explicit evidence-tier system (Section III), which at minimum prevents a code-reading hypothesis from being presented with unearned confidence, but it does not substitute for a second reviewer.

Curated, not exhaustive, tables. With 89 repositories, per-layer tables in Section IV present representative rather than exhaustive cases; Appendix A lists the full corpus, but a per-repository Layer 1/2/3/integrity classification for all 89 (rather than the curated examples in Section IV) has not yet been assembled into a single machine-readable table and remains future work.

Disclosure incomplete. As stated in Section X, this is both an ethical and a practical limitation: at the time of submission, two of the five findings have been formally reported and are awaiting maintainer response, and the remaining three lack a working formal disclosure channel and are being pursued through best-effort fallbacks. Most venues in this space expect responsible disclosure to be complete, or substantially underway with a documented good-faith attempt, prior to submission for a finding of this severity; none of the five had a confirmed remediation at the time of writing.

XII. CONCLUSION AND FUTURE WORK

Across 89 manually audited MCP servers, the dominant governance failure is architectural, not syntactic: the absence of a decision on what an LLM-driven caller should be allowed to do, expressed as an unscoped 1:1 mapping of an underlying API’s control plane onto MCP tools. Where containment is attempted and fails, we now distinguish at least three sub-types (sub-DSL injection, inconsistent control application, and unsound containment primitives), the last illustrated by a reproducible sandbox escape against a healthcare-data server; we stop short of claiming a comparative failure *rate* between attempted and absent containment, since the corpus does not yet support that statistic (Section XI). Where governance is done well, it is not a single pattern but a catalog of at least ten design choices actually observed in deployed code, four of them independently discovered in unrelated codebases, none of

which the ecosystem’s adoption signals (stars, vendor backing) appear to reliably select for.

This corpus was assembled to ground a broader, ongoing line of work on continuous authorization-policy hardening for agentic systems — a “dual-loop” framework in which a red loop probes both the existence and the correctness of a server’s claimed boundaries, and a blue loop treats “we built a mechanism” and “the mechanism is on by default and actually holds under the exact construction an attacker will try” as two separate, both-necessary claims. The corpus’s central finding speaks directly to why that separation matters: governance failure is overwhelmingly a design gap, not a code defect, and where a design decision *was* made, our containment-integrity sub-types show that a red loop verifying only “does a boundary exist” would have missed every case in Section VI — each one has a boundary that exists and fails anyway, for a different reason. Future work includes completing responsible disclosure for the findings reported here (Section X), a second-coder verification pass with reported inter-rater agreement on a stratified subsample, a registry-scale sampling frame to complement this corpus’s domain-stratified design, and scoring the full corpus against the severity model of Section IX.

OPEN SCIENCE

We will release, at the anonymized-artifact deadline, the static pattern scanner used for triage (Section III) and the proof-of-concept reconstruction scripts underlying the sandbox-escape (Section VI-A) and sub-DSL-injection (Section VI-C) findings; both scripts run against synthetic placeholder data and have no dependency on a live target. The full corpus listing (Table V, Appendix A) is included in this submission. Five repository names remain withheld pending completion of responsible disclosure (Section X); we will update the artifact and the camera-ready manuscript once that process concludes, per the disclosure policy stated there. Upon acceptance we will deposit the final artifact package, including the corpus listing, scanner, and PoC scripts, on Zenodo.

LLM USAGE CONSIDERATIONS

This research was conducted through an extended, interactive collaboration between the author and an AI research assistant (Claude, Anthropic). The assistant performed the majority of the audit’s mechanical work under the author’s direction and real-time review: cloning and reading candidate repositories, running the static pattern scanner and triaging its

output, drafting the per-round audit write-ups that this paper’s taxonomy and case studies are drawn from, constructing and executing the proof-of-concept scripts described in Section VI (including the sandbox-escape and sub-DSL-injection reconstructions), and producing successive drafts of this manuscript. The author selected the domains and repositories audited in each round, reviewed and approved every finding before it was recorded, decided the taxonomy and its extensions, and made the severity and disclosure judgment calls in Sections IX and X; the author is responsible for this paper’s content and conclusions. No finding reported here was accepted without the author’s review of the underlying code and, where applicable, the executed proof-of-concept output. We disclose the extent of this assistance in the interest of transparency about a practice that is, in our experience, already common in applied security research but not yet routinely documented, and we consider that disclosure itself a small contribution of this paper, independent of its taxonomy.

REFERENCES

- [1] J. H. Saltzer and M. D. Schroeder, “The Protection of Information in Computer Systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [2] N. Hardy, “The Confused Deputy: (or why capabilities might have been invented),” *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, 1988.
- [3] M. S. Miller, “Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control,” Ph.D. dissertation, Johns Hopkins University, 2006.
- [4] V. C. Hu et al., “Guide to Attribute Based Access Control (ABAC) Definition and Considerations,” NIST Special Publication 800-162, 2014.
- [5] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
- [6] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- [7] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
- [8] Invariant Labs, “MCP Security Notification: Tool Poisoning Attacks,” <https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks>, April 2025.
- [9] GitHub Security Advisory GHSA-g644-9gfx-q4q4 (CVE-2023-37466) and CVE-2023-37903, “Sandbox Escape in vm2,” patriksimek/vm2, 2023.
- [10] Node.js Foundation, “vm — Node.js Documentation: ‘The vm module is not a security mechanism’,” <https://nodejs.org/api/vm.html>, accessed 2026.
- [11] P. Runeson and M. Höst, “Guidelines for Conducting and Reporting Case Study Research in Software Engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [12] S. Gaire, S. Gyawali, S. Mishra, S. Niroula, D. Thakur, and U. Yadav, “Systematization of Knowledge: Security and Safety in the Model Context Protocol Ecosystem,” arXiv:2512.08290, 2025.
- [13] X. Li and X. Gao, “A First Look at the Security Issues in the Model Context Protocol Ecosystem,” arXiv:2510.16558, 2025. Accepted, IEEE/IFIP International Conference on Dependable Systems and Networks (DSN) 2026.
- [14] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, and A. E. Hassan, “Model Context Protocol (MCP) at First Glance: Studying the Security and Maintainability of MCP Servers,” arXiv:2506.13538, 2025 (revised 2026).
- [15] M. Rostamzadeh, S. Narula, N. Birhan, M. Ghasemigol, and D. Takabi, “MCP-DPT: A Defense-Placement Taxonomy and Coverage Analysis for Model Context Protocol Security,” arXiv:2604.07551, 2026.
- [16] Anthropic, “Model Context Protocol Specification,” <https://modelcontextprotocol.io>, accessed 2026.

APPENDIX

Table V lists all 89 distinct repositories in the corpus by audit round, star count at time of audit, and primary language. Star counts were not consistently recorded for rounds 1–5 (31 repositories; recording began systematically in round 6) and are marked “n/r” where unavailable — we report this gap rather than back-filling it from a later, potentially different star count. Five repository names are withheld, marked [REDACTED], pending completion of responsible disclosure (Section X); their round, star count, and language are shown. Two repositories received two audit passes each (Section III) and are listed once, at the round of their fullest audit, with a footnote noting the earlier pass.

Rnd	Repository	★	Lang.
1	modelcontextprotocol/servers (official)	n/r	multi
1	Sunwood-ai-labs/github-kanban-mcp-server	n/r	TS
1	Sunwood-ai-labs/documind-mcp-server	n/r	TS
1	Sunwood-ai-labs/aira-mcp-server	n/r	TS
1	box-community/mcp-server-box	n/r	Python
1	containers/kubernetes-mcp-server	n/r	Go
1	vitest-community/mcp	n/r	TS
2	efforthye/fast-filessystem-mcp	n/r	TS
2	QuantGeekDev/docker-mcp	n/r	Python
2	minhalvp/android-mcp-server	n/r	Python
2	inercia/mcp-cli-adapter	n/r	Go
2	kurdin/github-repos-manager-mcp	n/r	TS
2	pansila/mcp_server_gdb	n/r	Rust
3	ergut/mcp-bigquery-server	n/r	TS
3	QuantGeekDev/mongo-mcp	n/r	TS
3	Xexr/mcp-libsql	n/r	TS
3	kiliczsh/mcp-mongo-server	n/r	TS
3	strowk/mcp-k8s-go	n/r	Go
4	voska/hass-mcp	n/r	Python
4	rohit-kaundal/digitalocean-mcp-server	n/r	Go
4	browsermcp/mcp	n/r	TS
4	gologinapp/gologin-mcp	n/r	TS
4	isaacphi/mcp-language-server	n/r	Go
5	danfedick/vault-mcp	n/r	Go
5	jsyapps/stripe-mcp	n/r	TS
5	fabienbutz/smtp-mcp	n/r	TS
5	visusnet/coinbase-mcp-server	n/r	TS
5	vkdnjznd/crypto-trading-mcp*	n/r	TS
5	yahyamohmuedpro99/mailer-mcp*	n/r	TS
6	sshaaf/keycloak-mcp-server†	43	Java
6	mrz1880/mcp-keycloak-admin	1	TS
6	korotovskys/slack-mcp-server	1,800	Go
6	SaseQ/discord-mcp	468	Java
6	jaimenbell/discord-mcp	0	Python
6	chigwell/telegram-mcp	1,400	Python
7	hifriendbot/agentwallet-mcp	3	TS
7	ExpertVagabond/solana-mcp-server	1	TS/Rust
7	JamesANZ/evm-mcp	3	TS
7	lightningfaucet/lightning-wallet-mcp	8	TS
7	avisangle/jenkins-mcp-server	4	Python
7	hashicorp/terraform-mcp-server (official)	1,500	Go
8	domdomegg/airtable-mcp-server†	454	TS
8	rashidazarang/airtable-mcp	85	TS/JS
8	awkoy/notion-mcp-server	165	TS
8	nguyenvanduocit/jira-mcp	90	Go
8	ChrisGVE/workspace-qdrant-mcp	2	Rust
8	raalarcon9705/jira-mcp	5	TS
9	[REDACTED] (sub-DSL injection case, §VI-C)	134	Python
9	[REDACTED] (sub-DSL injection case, §VI-C)	4	Python
9	MarceauSolutions/md-to-pdf-mcp	8	Python
9	us-all/datadog-mcp-server	2	TS
9	utapyngo/sentry-mcp-rs	9	Rust
9	getsentry/sentry-mcp (official)	837	TS
10	mbelinky/x-mcp-server	21	TS
10	abhineet34/linkedin-mcp-server	2	TS

Rnd	Repository	★	Lang.
10	devag7/linkedin-mcp	1	TS
10	karanb192/reddit-mcp-buddy	801	TS
10	gwbischof/bluesky-social-mcp	16	Python
10	cameronrye/activitypub-mcp	17	TS
11	ofershap/mcp-server-s3	1	TS
11	isaacphi/mcp-gdrive	284	TS
11	hmk/box-mcp-server	11	TS
11	microsoft/playwright-mcp (official)	35,500	TS
11	co-browser/browser-use-mcp-server	843	Python
11	helbertparanhos/cloudflare-mcp-pro	2	TS
12	mendableai/firecrawl-mcp-server	7,400	TS
12	exa-labs/exa-mcp-server	4,900	TS
12	tavily-ai/tavily-mcp	2,400	TS
12	[REDACTED] (SSRF inconsistency case, §VI-D)	777	TS
12	apify/actors-mcp-server	5,800	TS
12	cablate/mcp-google-map	405	TS
13	[REDACTED] (anonymous-execution case, §VI-B)	167	TS
13	sanketskasar/hubspot-mcp-server	3	JS
13	reminia/zendesk-mcp-server	103	Python
13	intercom/intercom-mcp-server (official; no local code)	6	—
13	amir-bengherbi/shopify-mcp-server	16	TS
13	techspawn/woocommerce-mcp-server	98	TS
14	intuit/quickbooks-online-mcp-server (official)	343	TS
14	alpacahq/alpaca-mcp-server (official)	882	Python
14	arjabbar/PlaidMCP	1	C#
14	akutishevsky/lunchmoney-mcp	90	TS
14	XeroAPI/xero-mcp-server (official)	350	TS
14	calebl/ynab-mcp-server	116	TS
15	wso2/fhir-mcp-server	123	Python
15	[REDACTED] (sandbox-escape case, §VI-A)	82	TS
15	octalpixel/legal-context	0	TS
15	twentytwokhz/bamboohr-mcp	1	TS
15	alexmeckes/greenhouse-mcp	6	Python
15	calcom/cal-mcp (official)	22	TS

TABLE V: Full corpus, 89 distinct repositories across 15 audit rounds. *Scanned but not manually deep-audited (Section XI).
[†]Queued in round 4, deferred for lack of language support, fully audited in round 6 (shown once, at round 6). [‡]Also briefly examined in round 3 (shown once, at round 8, where fully profiled).